

Videclip

Projekthandbuch — Architektur, Prozesse & Ordnerstruktur

Version 1.0 · Mai 2026

Videclip — Projekthandbuch

Version: 1.0 · **Stand:** Mai 2026

Zweck: YouTube-Videos in virale Short-Form-Clips umwandeln (9:16, Untertitel, Hooks, Effekte).

1. Überblick

Videclip ist ein Full-Stack-Tool für Content-Creator:

- YouTube-URL einfügen
- KI findet die besten Highlight-Momente
- Automatischer Export als Shorts/TikTok/Reels mit eingebrannten Untertiteln, optional Cold-Open-Hook, Musik und Effekten

Komponente	Technologie
Frontend	React 18, Vite, Tailwind CSS
Backend	Node.js (ESM), Express
Highlight-Auswahl	Anthropic Claude (Sonnet)
Untertitel (STT)	Groq Whisper <code>large-v3</code>
Video	FFmpeg (ffmpeg-static), ffprobe
Download	yt-dlp / Piped / ytdl-core (konfigurierbar)

2. Projektstruktur

```
videclip/
├── client/ # React-Frontend (Dev: Port 5173)
│   ├── src/
│   │   ├── App.jsx # Haupt-UI, Analyze-Flow
│   │   ├── api.js # API-Client, Job-Polling
│   │   ├── components/ # HighlightCard, InputPanel, ...
│   │   ├── hooks/ # useLocalStorage
│   │   └── utils/ # previewQueue, helpers
├── server/ # Express-API (Port 3001)
│   └── src/
```

```
| | └─ index.js # Server-Start, Static, Cleanup
| | └─ config.js # Env, Prompts, Pfade
| | └─ routes/api.js # REST-Endpunkte
| | └─ services/ # Kernlogik (siehe Kapitel 4)
| | └─ lib/ffmpeg.js # FFmpeg-Pfad-Binding
| | └─ utils/ # YouTube, Cookies, Fehler
| └─ assets/music/ # Preset-Hintergrundmusik
| └─ scripts/ # z. B. wide_content_detect.py
└─ docs/
  └─ projekt-dokumentation/ # Dieses Handbuch + PDF
└─ package.json # Root: npm run dev
└─ cookies.txt # Optional: YouTube-Auth
```

Temporäre Job-Dateien

Pro Analyze-Job unter `%TEMP%/videclip/<jobId>/` :

Ordner/Datei	Inhalt
source.mp4	Heruntergeladenes Quellvideo
meta.json	Highlights, Render-Settings, URLs
raw-clips/	Rohschnitte pro Highlight
transcripts/<id>/	STT-Cache, WAV-Extrakte
clips/	Fertige Analyze-Clips mit Untertiteln
thumbs/	Thumbnails
previews/	480p-Preview nach UI-Änderungen

Alte Jobs werden nach ca. 30 Minuten gelöscht (`tempFiles.js`).

3. Benutzer-Flow (Frontend)

[YouTube-URL] → Analyze → Job-Polling → 5 Highlight-Karten

↓

Pro Karte: Trim, Hook, Caption-Style, Wide-Overlay, Webcam

↓

Auto-Preview (debounced) oder „Download Clip“ (Final-Export)

Wichtige UI-Komponenten

Datei	Funktion
InputPanel.jsx	URL, Mood, Musik, Aspect Ratio, Caption-Style (global: fire/bold/minimal)
HighlightCard.jsx	Video-Player, Einstellungen, Preview-Queue, Export
ProgressSteps.jsx	fetching → transcribing → analyzing → editing → ready
WideOverlaySelector.jsx	Gaming-UI-Overlay (Buch/Menu) manuell
WebcamSelector.jsx	Webcam-PiP-Position

Globale Einstellungen (App.jsx)

- captionStyle : Standard **fire** (orange/rot) für alle Clips eines Jobs
- aspectRatio : 9:16, 1:1, 16:9
- music , zoom , colorGrade , captions

4. Backend-Services (Hintergrundprozesse)

4.1 Analyze-Pipeline (`analyzePipeline.js`)

Einstieg: `POST /api/analyze` → `runAnalyzeJob(jobId, { url, mood, renderSettings })`

Phase 0 – Kontext

- └ `downloadVideo()`
- └ `probeVideoSource()` – Breite/Höhe
- └ `buildHighlightDetectionContext()`
 - └ YouTube-Transcript (Highlight-Suche, KEIN Burn-in)
 - └ Kapitel aus Beschreibung
- └ `analyzeHighlights()` – Claude wählt `candidate_id`

Phase 0b – Anreicherung

- └ `enrichHighlightsWithHook()` – Cold-Open, Peak, Cliffhanger
- └ `boostWeakHighlights()` – schwache Clips optimieren

Phase 1 – Schneiden (parallel, concurrency 3)

- └ `cutRawClipWithHook()` → `raw_<id>.mp4` + gemessene Hook-Dauer

Phase 2 – STT (sequentiell, concurrency 1)

- └ `transcribeClipAudio()` – Groq Whisper, Wort-Timestamps

Phase 3 – Rendern (sequentiell)

- └ `processClip()` – 2-Pass: Video → ASS-Burn → optional Musik

Wichtig: YouTube-Transcript dient nur der **Highlight-Erkennung**. Eingebrennte Untertitel kommen **immer** aus Groq STT pro Clip.

4.2 Export / Preview (clipExport.js , api.js)

Route	Ablauf
POST /api/preview	prepareExportRawClip() → ggf. Re-Cut + Re-STT → processClip(preview: true)
POST /api/clip	Wie Preview, volle Qualität + Musik
POST /api/download-all	ZIP aus clips/

prepareExportRawClip re-schneidet bei Trim-/Hook-Änderung und transkribiert neu wenn nötig.

4.3 Video-Rendering (ffmpeg.js – processClip)

3-Pass-Architektur (Caption-Stabilität auf Windows):

Pass 1 – Video-Composite

Crop (9:16) → Zoom → Color Grade → Wide-Overlay → Webcam-PiP

Audio: copy (kein loudnorm – Sync mit Untertiteln)

Pass 2 – Untertitel-Burn (captionPipeline.js)

ASS-Dateien (Wort-für-Wort + optional Hook-Overlay-ASS)

FFmpeg: -vf subtitles=dateiname.ass (relativer Pfad, cwd=workDir)

Pass 3 – Musik (nur Final-Export, nicht Analyze)

mixMusicOntoVideo() – Video copy, Audio mix

4.4 Cold-Open Hook (hookClip.js , intelligentHook.js , ffmpeg.js)

Cold-Open = kurzer Teaser am Anfang, dann Hauptclip von vorne.

Quellvideo

```

|
|→ Main-Teil schneiden (start_time ... end_time)
|
|→ Teaser extrahieren (Peak-Moment, Dauer hook_teaser_duration)
|  └ Re-Encode für exakte Länge (ffprobe → hook_teaser_measured_sec)
|
|→ concat: [Teaser] + [Main] → raw_<id>.mp4

```

Parameter	Bedeutung
hook_peak_time	Absoluter Zeitpunkt im Quellvideo für Teaser-Szene
hook_teaser_duration	Gewünschte Teaser-Länge (Slider, 0.5-4 s)
hook_teaser_measured_sec	Tatsächliche Länge nach FFmpeg (ffprobe)
hook	Cliffhanger-Text (Overlay, weiß)
hook_teaser_line	Gesprochene Zeile im Teaser (aus STT)

```

intelligentHook.js wählt Peak + Cliffhanger lokal aus YouTube-Transcript (ohne Extra-API-Tokens).

```

4.5 Untertitel (transcript.js , preprocess.js , captionAlign.js)

```

Groq Whisper large-v3 (verbose_json, word timestamps)

```

```

| parseGroqTranscription() – ein Wort pro Segment
| refineWordTimings() – +55 ms Display-Lag (kein aggressives RMS-Filter mehr)
| buildClipCaptionTimings()
└ Cache v11 in transcripts/<id>/transcription.json

```

Render:

```

prepareClipCaptionWords() → resolveCaptionTimings() → buildAssSubtitles()

```

```

Styles: bold | minimal | fire (+ Keyword-Highlight in captionKeywords.js)

```

Quality Gate: Mindest-Wortanzahl, Coverage; bei Fehlschlag Retry-Strategien (Sprache auto, plain audio).

4.6 Highlight-Kandidaten (`highlightCandidates.js` , `claude.js`)

- Lokale Kandidaten (~12 Fenster) mit Scores, dynamischer Länge (`dynamicClipLength.js`)
- Claude erhält nur `candidate_id` – erfindet keine Timestamps
- `highlightFromCandidate()` mappt zurück auf echte start/end

4.7 Weitere Services

Service	Aufgabe
<code>download.js</code>	Video + Metadaten von YouTube
<code>youtubeTranscript.js</code>	Transcript für KI
<code>smartCrop.js</code>	Gesichtserkennung (optional OpenCV)
<code>wideContentOverlay.js</code>	Gaming-UI-Erkennung (Python-Script)
<code>webcamPip.js</code>	Webcam-Bild-im-Bild
<code>music.js</code>	Preset-Tracks nach Mood
<code>renderSettings.js</code>	Aspect, Wide-Overlay auto bei 16:9
<code>clipBoost.js</code>	Schwache Clips: Trim, Hook, Effekte
<code>jobs.js</code>	In-Memory Job-Status
<code>previewQueue.js</code>	Server: eine Preview gleichzeitig
<code>cookieRefresh.js</code>	YouTube-Cookies erneuern

5. API-Referenz

Methode	Pfad	Beschreibung
GET	/api/health	Status + Musik-Presets
POST	/api/analyze	Start Analyze-Job → { jobId }
GET	/api/jobs/:id	Job-Status + Ergebnis
POST	/api/preview	Preview-Render (480p)
POST	/api/clip	Final-Export
POST	/api/download-all	ZIP-Download
POST	/api/upload-music	Custom MP3
POST	/api/cookies/refresh	YouTube-Cookies
GET	/api/files/:jobId/...	Clips, Thumbs, Previews

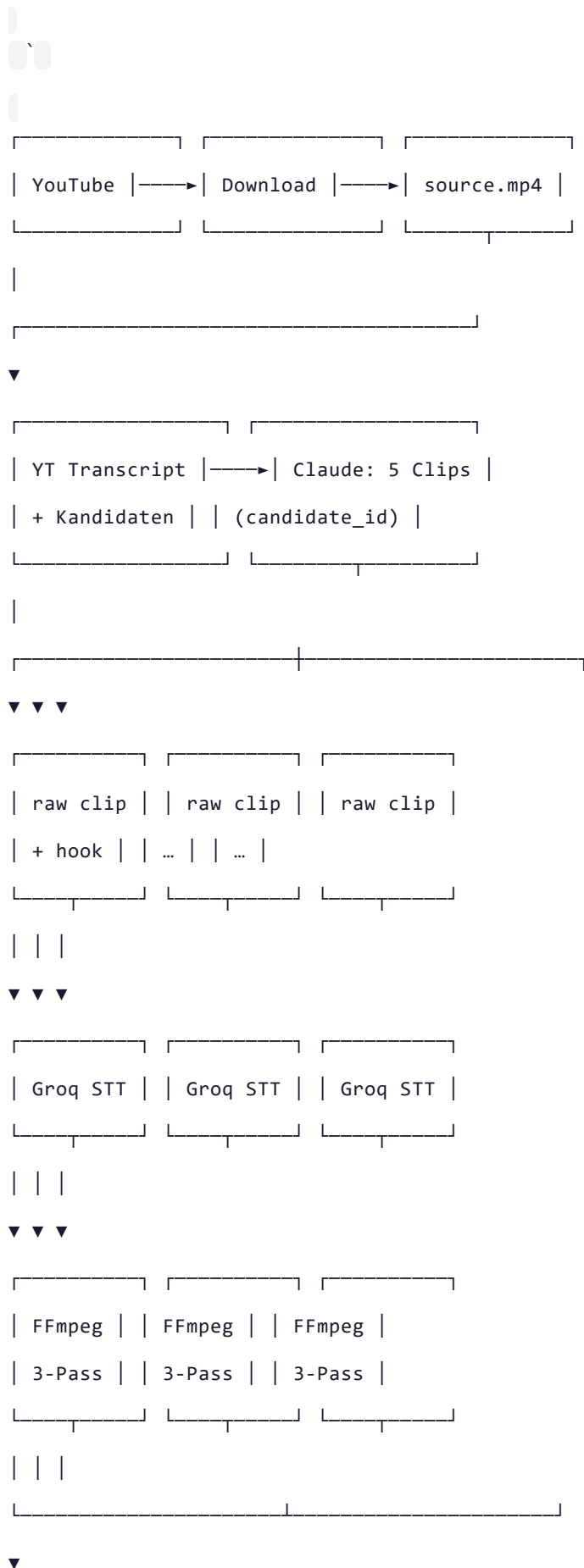
Job-Status

fetching → transcribing → analyzing → editing → ready (bzw. error)

6. Konfiguration (server/.env)

Variable	Bedeutung
ANTHROPIC_API_KEY	Claude für Highlight-Auswahl
GROQ_API_KEY	Whisper STT für Untertitel
CLAUDE_MODEL	z. B. claude-sonnet-4-6
PORT	Standard 3001
COOKIES_PATH	Netscape cookies.txt für YouTube
YTDLP_PATH	yt-dlp Executable
PYTHON_PATH	Für wide_content_detect.py

7. Datenfluss-Diagramm (Analyze)



meta.json + clips/*.mp4



React UI



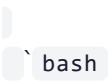
8. Bekannte Grenzen

Thema	Limitierung
Untertitel-Genauigkeit	Whisper bei 2+ Sprechern / Cross-Talk limitiert
Hook-Text vs. Sprache	Overlay (hook) ≠ Wort-Untertitel (STT)
Preview vs. Analyze	Preview 480p, ohne Musik; Analyze = Referenz-Qualität
Rate Limits	Groq 429 → Retry mit Backoff
YouTube	Cookies können ablaufen → /api/cookies/refresh

Geplante Verbesserungen (Roadmap)

- Speaker-Diarization / Hauptsprecher-Isolation
- YouTube-Transcript als Wort-Sync wo verfügbar
- A2-lite Jump-Cuts (Pacing)
- Caption-Editor in der UI

9. Entwicklung & Betrieb



Installation

npm run install:all

Entwicklung (Server + Client)

npm run dev

UI: <http://localhost:5173>

API: <http://localhost:3001>

Production Build

npm run build

npm start



Logs beobachten

Präfix	Bedeutung
[Pipeline]	Analyse-Phasen
[transcript]	Groq STT
[captions]	ASS-Erstellung / Fehler
[cut]	FFmpeg Schnitt, gemessene Hook-Dauer
[caption-align]	Timing-Lag
[intelligent-hook]	Hook-Peak + Cliffhanger

10. Datei-Index (Services)

Datei	Verantwortung
analyzePipeline.js	Gesamter Analyze-Job
claude.js	Claude API, Highlight-Parsing
highlightCandidates.js	Lokale Kandidaten-Generierung
highlightQuality.js	Confidence, Finalisierung
transcript.js	Groq STT, Cache, Quality Gate
ffmpeg.js	Schnitt, processClip, Hook-Concat
clipExport.js	Preview/Export-Vorbereitung
preprocess.js	ASS-Untertitel, Formatierung
captionPipeline.js	2-Pass ASS-Burn, Musik-Mix
captionAlign.js	Display-Lag auf Wort-Timestamps
hookClip.js	Cold-Open-Logik, Dauer-Helfer
intelligentHook.js	Teaser-Peak, Cliffhanger
coldOpenTiming.js	Mindest-Abstand Peak ↔ Clip-Ende
dynamicClipLength.js	15-22 s Clip-Länge
renderSettings.js	Globale Render-Optionen
captionKeywords.js`	Fire-Style Keyword-Betonung

Ende des Handbuchs – Videclip © Projekt videclip